Task 1: `Monkeys`

Authored by: Lim Rui Yuan

Prepared by: Pannawit Supanwassa

Editorial written by: Pannawit Supanwassa and Jayden Tiew

Subtask 1

Additional constraints: $n = 2$

There are only two monkeys. This implies that there is either only one pair of friends (the both of them) or none, hence the answer is either is 0 or 1.

For the answer to be 1, both monkeys must meet. To do so, they must either already be in the same position, or all these conditions must be met:

- If $p[1] < p[2]$, then $d[1] = \texttt{R}$ and $d[2] = \texttt{L}$
- Otherwise $d[1] = \texttt{L}$ and $d[2] = \texttt{R}$
- They must meet within k days. For this, $p[2] - p[1] \leq 2k$
- $p[1]$ and $p[2]$ must have the same parity, so that they do not "cross over" without meeting each other.

The proof that these conditions are necessary and sufficient will be shown in later subtasks.

Output either 0 or 1 depending on whether these conditions are met.

Time complexity: $\mathcal{O}(1)$

Subtask 2

Additional constraints: $d[1] = d[2] = \cdots = d[n]$

All monkeys are moving in the same direction. Notice that this means that the relative positions of the monkeys will never change (the leftmost monkey will always remain leftmost, the second leftmost will always remain second leftmost etc.)

Therefore, if two monkeys are not already in the same initial position, they will never meet. The answer is the number of pairs of monkeys that initially start in the same position. To count this value efficiently, we can use an `std::map` data structure.

Time complexity: $\mathcal{O}(n)$

Subtask 3

Additional constraints: $n, k \leq 200$

The small constraints allow us to simulate the process directly.

For all k days:

1. Move each monkey to the left or right accordingly, updating the entire position array p.
2. Iterate over all pairs of monkeys. If they lie at the same position, a new pair of friends is found and we increment our answer by 1.

Take extra care for monkeys that start at the same position, and avoid double-counting monkeys that start at the same position and have the same direction.

Time complexity: $\mathcal{O}(n^2k)$

Subtask 4

Additional constraints: $n, k \leq 3000$

We speed up the solution from subtask 3.

By storing the positions of the monkeys in a `std::map` instead, we can simply iterate over the map to determine the number of monkeys on the same position and count the number of such pairs across all positions.

Time complexity: $\mathcal{O}(nk \log n)$

Subtask 5

Additional constraints: $n \leq 3000$

We use the idea in subtask 2 to make an observation:

Lemma 1.1. *There are only two mutually excxlusive ways that a pair of monkeys can meet*

1. They both start on the same initial position
2. One monkey is on a position on the left moving towards the right, and the other is on a position on the right moving towards the left.

Proof. Consider two monkeys a and b that are on different positions (if they are, they are already a pair as mentioned in point 1). WLOG assume $p[a] < p[b]$

If they both move in the same direction (subtask 2), they will never meet. Hence the only other two ways are for $d[a] = \texttt{L}$, $d[b] = \texttt{R}$ and $d[a] = \texttt{R}$ and $d[b] = \texttt{L}$.

However it is easy to see that in the former case, the pair moves away from each other and hence never meet. In the latter case, they approach, hence being the only possible way they can meet, completing the proof. □

As they are both mutually exclusive cases, we can count pairs of case 1 easily using a `std::map` like in subtask 2.

What remains is counting the pairs for the 2nd case. We can actually make use of ideas from subtask 1.

Lemma 1.2. *For a pair of monkeys that are moving towards each other, another necessary condition for them to meet is for the parities of their positions to be the same.*

Proof. Consider two monkeys where the parities of their positions are different (one is odd and one is even), their parities will always remain different throughout the k days. This is because the positions of both monkeys will always increase/decrease by 1, implying their parities will also both change everyday. Hence both monkeys will never meet on the same position, as that would obviously require an equal parity. □

Lastly, the distance between the two monkeys must be at most $2k$ for them to meet.

Hence, we can simply extend the solution from subtask 1 by iterating over all pairs of monkeys that satisfies all conditions, as well as combining it with subtask 2 to count monkeys of equal positions.

Note the edge case of monkeys already being on the same position and and moving in opposite in directions. These pairs could be double counted as they fulfil both conditions of case 1 and 2 in theorem 1.1. To resolve this, we separate right-moving and left-moving monkeys on the same position.

Time complexity: $\mathcal{O}(n^2)$

Subtask 6

Additional constraints: None

To solve the full problem, we need a more efficient way to count pairs that satisfy the conditions found.

Let's sort the positions of the monkeys. Then for any monkey i with $d[i] = \texttt{R}$, our task becomes finding a way to quickly find the number of monkeys j to the right of i with $d[j] = \texttt{L}$ and $p[j] - p[i] \leq 2k \implies p[j] \leq 2k + p[i]$

Notice that if we sweep over all right-moving monkeys from left to right, notice that the right-most monkey j that satifies the conditions monotonically increases (i.e. as $2k + p[i]$ increases, the largest $p[j]$ also increases)

This allows us to apply the classic technique of sliding window to solve the task. As we sweep through right moving monkeys from left to right, maintain a sliding pointer that points to the rightmost monkey j, sliding it as necessary as we sweep, and carefully adding the number of monkeys within our window to our answer at each step.

Note that we also have to consider the parity of the monkeys. The easiest way to do this is to split all monkeys into even and odd positions, and then perform the same algorithm for both groups of monkeys separately (as they are independent of each other)

Time complexity: $\mathcal{O}(n \log n)$

Task 2: `Famished Cats`

Authored by: Seah E-ket

Prepared by: Zhao Yaoqi and Seah E-ket

Editorial written by: Ziv Lim

Subtask 1

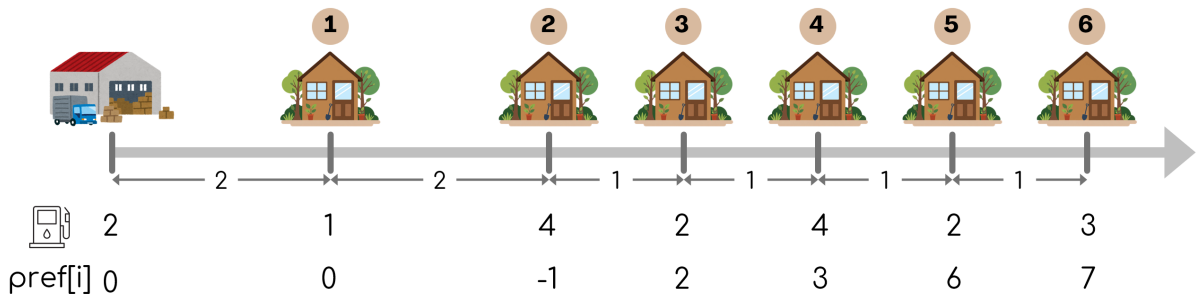
Additional constraints: $|f[i] - f[n-i+1]| = k$ for some constant k, for all $1 \le i \le n$

For brevity, let $L = i$ and $R = n - i + 1$ for some $i \le \frac{n}{2}$.

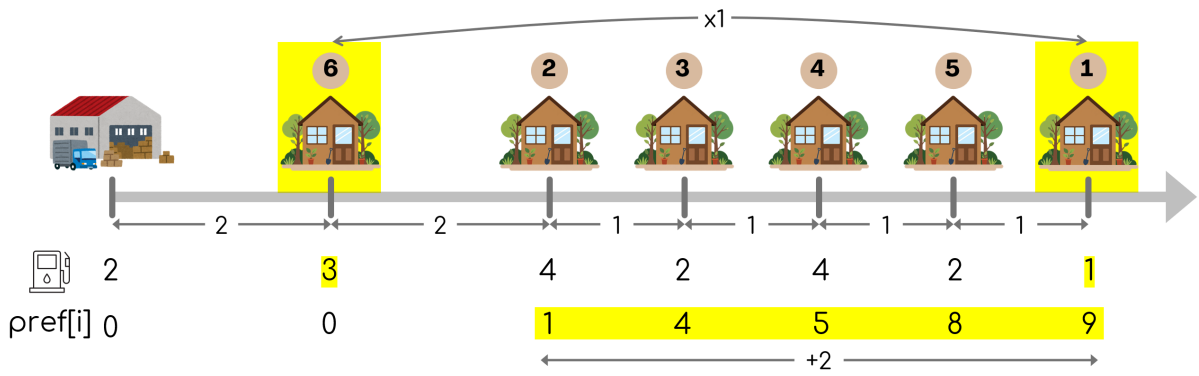
Lemma 2.1. *In any optimal solution, we only ever swap* (L, R) *when* $f[R] > f[L]$.

Proof. Suppose $pref[i]$ represents fuel remaining after arriving at house i, before taking the fuel at i. The truck can reach house i if and only if $pref[i] \ge 0$. Swapping (L, R) adds $f[R] - f[L]$ to $pref[L+1, R]$. It is clear that if $f[R] < f[L]$, it only makes things worse—so we would never swap in this case.

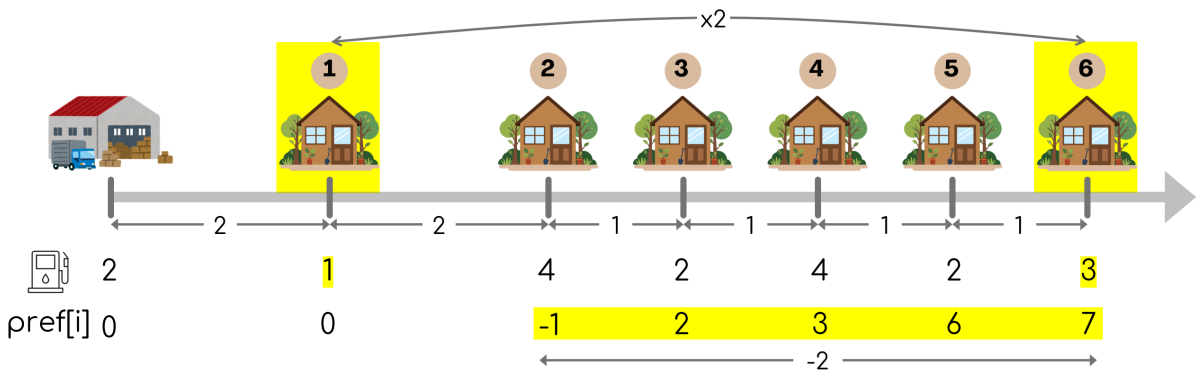
For visualisation purposes, consider the test case:
$n = 6, x = 2$
$d = \{2, 2, 1, 1, 1, 1\}$
$f = \{1, 4, 2, 4, 2, 3\}$



If you swap (1, 6), it's akin to adding $f[6] - f[1] = +2$ for $pref[2, 6]$, making fuel stops more easily accessible.



If you undo the swap, it's akin to adding $f[1] - f[6] = -2$ for $pref[2, 6]$, making fuel stops harder to access, as we are running on less fuel.



Hence, we should only swap (L, R) when $f[R] > f[L]$ in the optimal solution. □

Lemma 2.2. *If the fuel gain is the same, i.e., $f[R_1] - f[L_1] = f[R_2] - f[L_2]$, where $L_1 < L_2$, it is optimal to prioritise swapping (L_1, R_1) over (L_2, R_2).*

Proof. By Lemma 2.1, swapping (L_1, R_1) adds $f[R_1] - f[L_1]$ to $pref[L_1 + 1, R_1]$. Since $L_1 < L_2$, this range is strictly larger than $[L_2 + 1, R_2]$, meaning the swap at (L_1, R_1) improves at least as many prefix sums as the swap at (L_2, R_2). Therefore, it is always at least as beneficial to swap the earlier pair first. □

For simplicity, we present the $\mathcal{O}(n \log n)$ solution instead of the $\mathcal{O}(n)$ solution.

Finding D: By Lemma 2.1, we always swap (L, R) when $f[R] > f[L]$. Performing all such beneficial swaps maximises every prefix sum, giving the optimal distance D.

Finding S: Recall that due to subtask constraints, a swap (L, R) adds $\pm k$ to $pref[L+1, R]$. By Lemma 2.2, it is always optimal to prioritise swapping earlier pairs first, as each beneficial swap contributes the same gain k. Binary search on S by swapping the first S valid pairs (i.e., the S earliest pairs where $f[R] > f[L]$) and checking if D is still reachable.

Time Complexity: $\mathcal{O}(n \log n)$

Subtask 2

Additional constraints: $n \leq 40$

Notice that swapping (L, R) twice reverts to the original—so each pair is either swapped or not. We can represent each of the $\frac{n}{2}$ pairs as a bit: 1 if swapped, and 0 if not swapped.

With this, we brute force all $2^{\frac{n}{2}}$ combinations. For each bitmask, simulate the journey and track the furthest house reached. Among all masks that achieve the furthest house D, take the one with the fewest bits set as S.

Time Complexity: $\mathcal{O}(2^{\frac{n}{2}})$

Subtask 3

Additional constraints: f is non-decreasing ($f[i] \leq f[i+1]$ for all $1 \leq i \leq n-1$)

Since f is non-decreasing, earlier pairs always yield a greater or equal fuel gain than later pairs. Consider $L_1 < L_2$ (and thus $R_1 > R_2$):

$$f[L_1] \leq f[L_2] \quad \text{and} \quad f[R_1] \geq f[R_2]$$
$$\implies f[R_1] - f[L_1] \geq f[R_2] - f[L_2]$$

So earlier pairs not only cover a larger range of prefix sums (Lemma 2.2), but also provide a greater or equal fuel boost. The greedy from Subtask 1 applies directly without modification.

Subtask 4

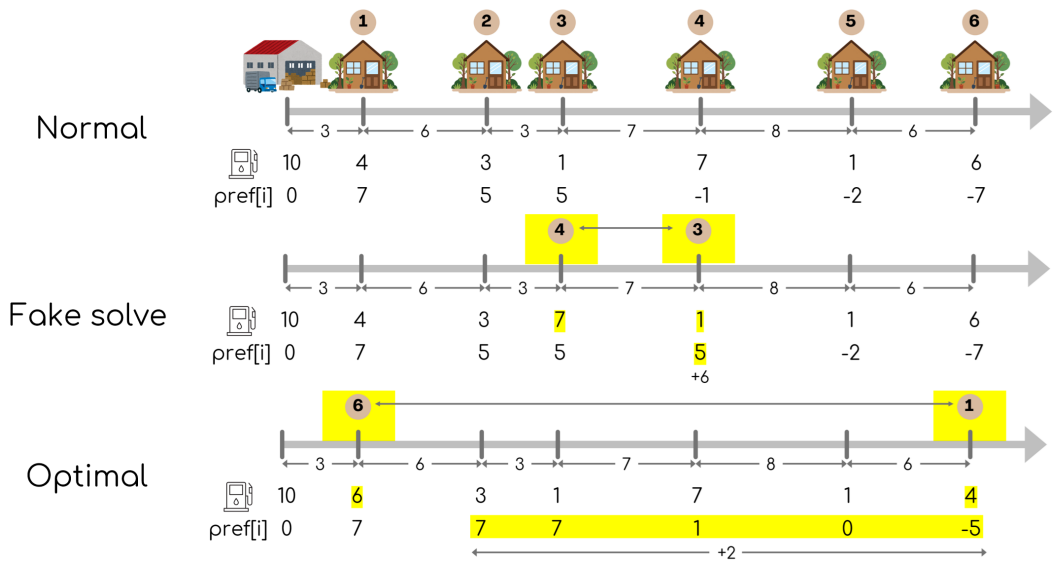
Additional constraints: $D \leq \dfrac{n}{2}$

This subtask is designed to catch a common "fake solution": greedily picking the swap with the largest fuel gain at the moment you need it, using a priority queue. When fuel drops below 0 at some house, pop the highest-gain swap available from the priority queue and apply it.

However, this does not account for the fact that fuel boosts are temporary: while the boost works where $i \leq \frac{n}{2}$, it may not work after that (i.e., the range add is only from $[L+1, R]$ and does not span the entire range).

Consider the counterexample:
$n = 6, x = 10$
$d = \{3, 6, 3, 7, 8, 6\}$
$f = \{4, 3, 1, 7, 1, 6\}$

The available swaps are:
$(L = 1, R = 6)$: $gain = f[6] - f[1] = 6 - 4 = 2$, boosts $pref[2, 6]$
$(L = 3, R = 4)$: $gain = f[4] - f[3] = 7 - 1 = 6$, boosts $pref[4, 4]$



At index 4, $pref[4] = -1 < 0$, so the fake solution swaps $(3, 4)$ to gain 6 fuel.
Then at index 5, $pref[5] = -2 < 0$, so it swaps $(1, 6)$ to gain 2 fuel and reach $D = 5$.

However, only one swap is needed: swapping $(1, 6)$ alone makes both $pref[4] > 0$ and $pref[5] > 0$, reaching $D = 5$.

Subtask 5

Additional constraints: $n \leq 5000$

Finding maximum distance D: Already shown in Subtask 1.

Finding minimum swaps S: Since inner swaps do not affect outer positions, we can use DP processing pairs inside-out. Let dp$[i][j]$ = minimum fuel needed at position $L = i$ to reach $\min(D, R)$, using j swaps among pairs $[i, \frac{n}{2}]$.

We present only the gist of the solution, as Subtask 6 is easier to explain.

Time Complexity: $\mathcal{O}(n^2)$

Subtask 6

Additional constraints: None

Finding minimum swaps S: We process pairs from the outermost ring inward (looping from $L = 1, R = n$ to $L = \frac{n}{2}, R = \frac{n}{2} + 1$).

The key observation is that inner swaps do not affect the prefix sums of outer positions. So when processing ring (L, R), all swaps currently in the priority queue are from outer rings, whose boost range already covers position L. This means any swap popped from the queue is guaranteed to remain in effect at the current and all future positions—so we can maintain a simple offset variable representing the total fuel delta from all swaps taken so far, and this offset does not expire.

This allows us to safely use a max-priority queue: at any point where $pref[L] + \text{offset} < 0$, popping the largest available gain until $pref[L] + \text{offset} \geq 0$ is optimal, since every swap in the queue boosts the current and all remaining inner positions anyway.

Two cases arise when processing ring (L, R):

- $R > D$ (R is inaccessible): Only L matters. Fix any negative $pref[L] + \text{offset}$ by popping from the priority queue, then push $f[R] - f[L]$ (if positive) once $pref[L] + \text{offset} \geq 0$.
- $R \leq D$ (both L and R are accessible): Both $pref[L] + \text{offset}$ and $pref[R] + \text{offset}$ must be non-negative, so that we can reach both positions. Fix using the priority queue as needed, remembering to push $f[R] - f[L]$ (if positive) once $pref[L] + \text{offset} \geq 0$.

Time Complexity: $\mathcal{O}(n \log n)$

Alternative Solution for Subtask 6

Finding S: We find the minimum number of swaps needed to still reach D. Starting from the state where all valid pairs are swapped, greedily unswap pairs in increasing order of loss (i.e., increasing $f[R] - f[L]$), tie-breaking by smaller range first. A pair can be unswapped if, after the unswap, $pref[1, D] \geq 0$; otherwise, skip it.

Theorem 2.3. *Undoing swaps in increasing order of loss, tie-breaking by shorter range first, is optimal.*

Proof. Since mirror pairs satisfy $L_1 < L_2 \implies R_1 > R_2$, all pair ranges are strictly nested.

Define $\text{loss}(L, R) = f[R] - f[L]$, in a way that unswapping (L, R) subtracts $f[R] - f[L]$ from $pref[L + 1, R]$.

Consider two swaps (L_1, R_1) processed before (L_2, R_2), meaning $\text{loss}(L_1, R_1) \leq \text{loss}(L_2, R_2)$, tie-breaking by shorter range. We show that keeping (L_2, R_2) is always at least as good. Since this holds for any adjacent pair in the ordering, global optimality follows by a standard exchange argument.

Since all ranges are nested, only two cases exist:

Case 1: (L_1, R_1) nested inside (L_2, R_2). Then (L_2, R_2) has larger range and greater or equal loss; processing (L_1, R_1) first is strictly better.

Case 2: (L_2, R_2) nested inside (L_1, R_1). Since (L_1, R_1) has the larger range, this case requires $\text{loss}(L_1, R_1) < \text{loss}(L_2, R_2)$ strictly. Two subcases arise:

- **Subcase A:** (L_1, R_1) is successfully unswapped.

```
L1 ================ R1
     L2 ====== R2
   Lk ============ Rk
            ....
Lx ========================= Rx
```

  Now consider any swap (L_k, R_k) **to be processed later and nested inside** (L_1, R_1): it has strictly greater loss over a strictly smaller range. If (L_k, R_k) cannot be unswapped, the failure lies entirely within $pref[L_k + 1, R_k]$.

  Since $[L_k + 1, R_k]$ is contained in $[L_1 + 1, R_1]$ and $\text{loss}(L_k, R_k) > \text{loss}(L_1, R_1)$, swapping their processing order—unswapping (L_k, R_k) while keeping (L_1, R_1)—would remove more fuel from exactly the positions that are already failing. Thus the original order is at least as good.

For any swap (L_x, R_x) **to be processed later that contains** (L_1, R_1): it must have strictly greater loss, so by Case 1, keeping (L_x, R_x) is already proven optimal.

- **Subcase B:** (L_1, R_1) cannot be unswapped—it stays swapped. Unswapping (L_2, R_2) does not affect the outer ranges $[L_1 + 1, L_2]$ and $[R_2 + 1, R_1]$, which remain covered by (L_1, R_1).

  Since (L_1, R_1) remains swapped regardless of processing order, the relative ordering between (L_1, R_1) and (L_2, R_2) has no effect on the final answer.

In all cases, processing in the prescribed order is optimal. □

Every swap and check can be performed using a segment tree with range add and range min queries in $\mathcal{O}(\log n)$.

Time Complexity: $\mathcal{O}(n \log n)$

Task 3: `3 Raptors`

Authored by: Sun Beichen

Prepared by: Ryan Goh Choon Aik

Editorial written by: Ryan Goh Choon Aik

Subtask 1 & 2

Additional constraints: $n \leq 500$ (Subtask 1), $n \leq 2000$ (Subtask 2)

A simple $\mathcal{O}(n^3)$ brute force suffices for Subtask 1. For subtask 2, optimise the brute force using prefix sums or some form of sliding window.

Time complexity: $\mathcal{O}(n^3)$ (Subtask 1), $\mathcal{O}(n^2)$ (Subtask 2)

Subtask 3 & 5

Additional constraints: $c[i] \leq 2$ (Subtask 3), there exists a $1 \leq j \leq n$ such that $c[i] \neq 3$ for all $i \leq j$, and $c[i] = 3$ for all $i > j$ (Subtask 5)

Since $c[i] = 3$ does not exist, the least common raptor colour is always zero. Hence, this problem reduces to finding the longest subarray such that the count of the other two colours are at most k. This can be easily done using a simple binary search, and sliding window technique.

Subtask 5 is just an extension of Subtask 3. We first use Subtask 3 solution on the subarray that does not contain any 3s. Then, we try all possible subarrays that end at j. For each subarray, we compute the number of 3s that can be added after j such that the criterion is still satisfied. Using a sliding window on the left pointer moving away from j, and keeping track of the counts of 1s and 2s, we can solve this portion in $\mathcal{O}(n)$ time.

Time complexity: $\mathcal{O}(n \log n)$

Subtask 4

Additional constraints: $k = 0$.

This problem reduces to finding the longest subarray such that the counts of all three colours

are the same.

WLOG let the most common raptor colour be colour 1 (this will be important in later subtasks). Consider an array x where $x[i] = 1$ if $c[i] = 1$, $x[i] = -1$ if $c[i] = 2$, $x[i] = 0$ if $c[i] = 3$. Let array p be the prefix sum array of x. Consider also an array y where $y[i] = 1$ if $c[i] = 1$, $y[i] = 0$ if $c[i] = 2$, $y[i] = -1$ if $c[i] = 3$. Let array q be the prefix sum array of y.

Observe that if you pick two l, r such that $p[l] = p[r]$, then the subarray $l + 1$ to r has equal numbers of colour 1 and colour 2 raptors. The logic similarly applies to array q.

Hence, it suffices to find 2 integers l and r ($l < r$ such that $p[l] = p[r]$, and $q[l] = q[r]$, such that the length of the subarray $r - l$ is the largest. We can do so by storing a map of pairs $(p[i], q[i])$ to i, keeping the largest and smallest possible i for each pair.

Time complexity: $\mathcal{O}(n \log n)$

Subtask 6 & 7

Additional constraints: In any contiguous sequence of 3 or more raptors, colour 3 is the least common (Subtask 6), No additional constraints (Subtask 7).

We define x, p, y, q similarly as in Subtask 4. Now, we want to find l and r such that $0 \leq (p[r] - p[l-1]) \leq k$ (Condition 1) and $0 \leq (q[r] - q[l-1]) \leq k$ (Condition 2).

We attempt to fix one of the conditions first. This can be done through sorting the array p, keeping track of the original indices i in the array $orig$. Then, for each y in p_{new}, we want to find the lowest x such that $p_{new}[y] - p_{new}[x-1] \leq k$. Now, condition 1 is always achieved; moreover, we can also fix the r in condition 2 by letting $r = orig[y]$.

Now, the problem simplifies to the following: We have $orig[x], orig[x+1], \ldots, orig[y-1], orig[y]$; and we want to find the minimum possible $orig[t]$ in this list such that $0 \leq (q[r] - q[orig[t]-1]) \leq k$, i.e. $q[r] - k \leq q[orig[t]-1] \leq q[r]$. This can be done using a segment tree that maps $q[i]$ to i when i is active in the sliding window, and ∞ otherwise. Our desired minimum possible $orig[t]$ will be a range query from $q[r] - k$ to $q[r]$ in our segment tree.

Do note that we will need to deal with same $q[i]$, which can be trivially done using a monotonic deque for each $q[i]$. Also, one must be cautious when implementing this solution due to the possible negative values involved in p and q.

Finally, since we assumed that the most common raptor colour is colour 1 in the solution, we will need to repeat the entire solution 2 more times assuming the most common colour is colour 2 and colour 3 respectively.

Subtask 6 is for solutions that failed to repeat the solution 3 times for each colour, or for other buggy solutions.

Time complexity: $\mathcal{O}(n \log n)$

Task 4: `Gemstones`

Authored by: Sun Beichen

Prepared by: Yaw Chur Zhe

Editorial written by: Yaw Chur Zhe

In this editorial, we use the notation $c[l \dots r]$ to refer to the subarray of gemstones starting at gemstone l and ending at gemstone r. That is, the subarray $c[l], c[l+1], \dots, c[r]$.

Subtask 1

Additional constraints: $c[1] = c[2] = \cdots = c[n]$

If there are an even number of gemstones, then it is always possible to delete all of them. If there are an odd number of gemstones, it is always possible to delete all but one of them.

Time complexity: $\mathcal{O}(n+q)$

Subtask 2

Additional constraints: Gemstones of the same colour form a contiguous subarray

From Subtask 1, we know that even-length monochromatic sequences will be deleted completely, whereas odd-length monochromatic sequences will leave one gemstone.

We can use prefix sums to count the number of odd-length monochromatic sequences in $c[l \dots r]$. The monochromatic sequences starting at l and ending at r will have to be processed separately.

Time complexity: $\mathcal{O}(n+q)$

Subtask 3

Additional constraints: $n, q \leq 2000$

Observe that the order of deletions does not matter.

For the j-th query, consider the gemstones from left to right from $l[j]$ to $r[j]$. Use a stack to maintain the undeleted gemstones: if the current gemstone matches the colour of the gemstone on top of the stack, pop the stack. Otherwise, push the current gemstone onto the stack. The answer for the j-th query is the size of the stack remaining.

Time complexity: $O(nq)$

Subtask 4

Additional constraints: $l[j] = 1$ for all $1 \leq j \leq q$

As per Subtask 3, use a stack to process the gemstones from left to right (starting from gemstone 1). Store the sizes of the stacks at each index, and output the answers accordingly in $\mathcal{O}(1)$ time for the q queries.

Time complexity: $\mathcal{O}(n + q)$

Subtask 5

Additional constraints: There are exactly two gemstones of each colour

Call a range of gemstones *deletable* if it is possible to perform deletions in the range such that no gemstones are left behind.

Consider the two gemstones x and y of a certain colour. x and y can be deleted if and only if the range of gemstones $c[x + 1 \ldots y]$ is deletable. By processing all gemstones with a stack (as per the previous two subtasks), we can identify all pairs of gemstones that can be deleted.

The number of gemstones deleted in a query is then twice the number of pairs of gemstones that can be deleted and are contained in that range. This can be computed with a range query data structure such as a Fenwick tree.

Time complexity: $\mathcal{O}((n + q) \log n)$

Subtask 6

Additional constraints: $c[i] \leq 2$ for all $1 \leq i \leq n$

Let us define the *reduced* form of a sequence to be the sequence of gemstones remaining after the sequence has been processed with the stack algorithm from Subtask 3.

Since $c[i] \in \{1, 2\}$, the reduced form of the sequences in this subtask will be of the form $121212\ldots$ or $212121\ldots$ (alternatingly).

We can use a segment tree to answer queries. Each node of the segment tree will store the reduced form of the range of gemstones it covers. When merging two segment tree nodes, there are a few cases to consider:

1. If the last gemstone of the left child node and the first gemstone of the right child node have the same colour, a number of deletions will occur. The number of pairs deleted is equal to the smaller of the two lengths of the reduced sequences.
2. Otherwise, no deletions occur.

To store information on the reduced sequence, it suffices to store its length and the colour of the first gemstone in the sequence.

Time complexity: $\mathcal{O}((n + q) \log n)$

Subtask 7

Additional constraints: $n, q \leq 100\,000$

We can use Mo's algorithm to solve this subtask. Mo's algorithm is a general approach for handling multiple queries on ranges, based on ideas from square-root decomposition[1].

Maintain a deque of undeleted gemstones. When extending the range by one (on either side), pop from the deque if the respective end of the deque has a gemstone of the same colour. Otherwise, push to that end of the deque.

When shrinking the range by one (on either side), if the gemstone to be removed has not already been deleted, simply pop it from the deque. Otherwise, a gemstone will have to be added to the deque. Beware of an edge case when there are three monochromatic gemstones in a row.

Hence, every expansion and shrinking of the range can be done in constant time.

Time complexity: $\mathcal{O}((n + q)\sqrt{n})$

Subtask 8

Additional constraints: $n, q \leq 300\,000$

[1] Refer to this article for a detailed explanation.

To illustrate the technique used in this subtask, let us first assume that all queries cross the midpoint $m = \lfloor \frac{n}{2} \rfloor$ of the given array. That is, $l[i] \leq m < r[i]$ for all $1 \leq i \leq q$.

Let us define the *reduced* form of a sequence as per Subtask 6. If we had the reduced forms of $c[l \ldots m]$ and $c[m+1 \ldots r]$, then we need only concern ourselves with deletions that occur at the point where these two reduced sequences merge (using the observation that the order of deletions does not matter).

The reduced forms of $c[m \ldots m], c[m-1 \ldots m], \ldots, c[1 \ldots m]$ can be precomputed by applying the stack algorithm from Subtask 3 in reverse, starting from gemstone m. However, storing all of these sequences explicitly would be infeasible. Instead, we consider a succinct tree-like representation that encodes all of these sequences:

- Create a tree with one node, which we will call ε.
- Let the *current node* $u := \varepsilon$.
- Process the gemstones starting from m and ending at 1. Let i be the index of the current gemstone.
  1. If $u = \varepsilon$ or the label of u is not equal to $c[i]$, create a child of u with label $c[i]$. Then set u to be the newly created child.
  2. Otherwise, set u to be its parent.
- In step 1 above, if u already has a child of label $c[i]$, set u to be that child without creating a new node.

Let $u[i]$ be the value of u just after the i-th gemstone was processed. This tree encodes all reduced sequences in the sense that the labels of nodes on the path from ε to $u[i]$ are exactly the reduced form of $c[i \ldots m]$.

Now, we will generate the tree for sequences starting at $m+1$ in the same way. However, instead of beginning with a tree of only one node, begin with the tree that was generated previously for sequences ending at m.

Let $v[i]$ be the current node (value of u) just after the i-th gemstone was processed. Similarly, the labels of nodes on the path from ε to $v[i]$ are exactly the reduced form of $c[m+1 \ldots i]$.

To answer a query from l to r, we observe the following. The nodes on the path from ε to $\mathrm{lca}(u[l], v[r])$ are exactly the gemstones deleted when merging the two reduced sequences $c[l \ldots m]$ and $c[m+1 \ldots r]$, where lca denotes the lowest common ancestor. Hence, the answer to the query is the length of the path from $u[l]$ to $v[r]$. This can be computed by applying standard algorithms such as binary lifting.

Now, let us relax the assumption that all queries cross the midpoint. To do so, we use a divide-and-conquer algorithm that recurses into the left and right halves of the array[2].

Time complexity: $\mathcal{O}(n \log^2 n + q \log n)$

Subtask 9

Additional constraints: None

Let τ be the tree generated when applying the algorithm in Subtask 8 once, starting from the first gemstone. Let $u[i]$ be the value of u just after the i-th gemstone was processed (define $u[0] = \varepsilon$ for convenience).

Surprisingly, the answer to the query (l, r) is equal to the length of the path from $u[l-1]$ to $u[r]$ in τ.

To prove this observation, let us consider $u[l-1 \ldots r]$. For all $l \leq i \leq r$, $u[i]$ is either the parent or a child of $u[i-1]$.

If there are two indices $l - 1 \leq i < j \leq r$ such that $u[i] = u[j]$, then $u[i+1 \ldots j]$ does not appear in the reduced sequence $c[l \ldots r]$[3]. By removing these subarrays from $u[l-1 \ldots r]$, we are left with a sequence s of distinct values that begins with $u[l-1]$ and ends with $u[r]$. The length of s is the answer to the query.

Further observe that s is precisely the nodes on the path from $u[l-1]$ to $u[r]$ in τ. Hence, the length of s is the length of the path from $u[l-1]$ to $u[r]$ in τ.

Again with standard algorithms, each query can be answered in $\mathcal{O}(\log n)$ time with an $\mathcal{O}(n \log n)$ precomputation.

Time complexity: $\mathcal{O}((n + q) \log n)$.

[2]Refer to this article for a detailed explanation.

[3]This is because a path that loops back to the same node corresponds to a sequence of elements that perfectly cancel each other out in the stack.

Task 5: `Lemon`

Authored by: Sun Beichen

Prepared by: Sun Beichen

Editorial written by: Zhao Yaoqi

Preliminary Observation

Lemma 5.1 (Variable-Length Encoding)**.** *If we have a solution where Takina **always** writes a binary string b of exactly length k, we can reduce this to $X \leq k-1$ by using variable-length encoding. Utilising all binary strings of lengths $0, 1, 2, \ldots, k-1$ provide $2^0 + 2^1 + \cdots + 2^{k-1} = 2^k - 1$ distinct encodings, which suffices to encode almost all length k binary strings.*

This optimisation applies to all solutions below. The stated X values are given **before** applying this lemma.

A summary of solutions is as follows:

Subtask	Solution	X	Y	X + Y
1	1	4500	1	4501
($Y \leq 2$)	2	288	2	290
2	3	81	7	88
($Y \leq 9$)	4	65	8	73
	5	49	8	57
	6	45	8	53
	7	44	8	52
	8	41	8	49
	9	31	8	39
3	10	18	42	60
(Minimize	11	18	30	48
$X + Y$)	12	27	14	41
	13	15	17	32
	14	14	17	31
	15	4	17	21

Subtask 1

Scoring: If $Y > 2$, score is 0. Otherwise, score is $10 \times \min\left(\frac{288}{X}, 1\right)$.

We are trying to minimise X while eating at most 2 fruits.

Solution 1: $X = 4500$**,** $Y = 1$

Constructing b:

Takina simply sends the entire permutation p through the bit string b. The bit string will contain $p[1]$ in binary, followed by $p[2]$ in binary and so on. Each $p[i]$ requires $\lceil \log_2 N \rceil = 9$ bits, so the total string length is $9 \times 500 = 4500$ bits.

This achieves $X = 4500$.

Eating fruits:

For the eating phase, Takina eats the fruit immediately *after* the lemon in the permutation order (if the lemon is the last fruit, she eats nothing). Hence $Y = 1$.

Recovering lemon:

Since Chisato knows the whole array p from the bit string b, she can identify the fruit immediately before the eaten fruit as the lemon (If no fruits are eaten, the last fruit is the lemon).

Solution 2: $X = 288$**,** $Y = 2$

Constructing b:

Instead of sending the entire permutation, Takina sends only the positions of the last 32 fruits, namely $p[469], p[470], \ldots, p[500]$. Encoding these 32 labels requires $9 \times 32 = 288$ bits.

This achieves $X = 288$.

Eating fruits:

If the lemon appears before the last 32 fruits:
There are 468 possible fruits for the lemon. Takina eats a pair of fruits from the last 32 fruits to encode the label of the lemon. There are $\binom{32}{2} = 496$ possible ways to eat 2 fruits from the last 32 fruits. For a specific pair of fruits eaten, let them have labels x, y, say a is the smaller of the two and b is the larger of the two. We then order these 496 ways in increasing order according

to the pair (a, b). Now, if the lemon's label is the k-th smallest amongst the first 468 fruits, we eat a pair of fruits in the last 32 so that (a, b) is the k-th smallest pair amongst all 496 ways.

If the lemon is among the last 32 fruits:
Takina eats the fruit immediately after the lemon (or nothing if the lemon is the very last fruit).

This achieves $Y = 2$.

Recovering lemon:

Chisato uses b to recover the labels of the last 32 fruits. If she sees 2 eaten fruits, she deduces the rank k of the pair (a, b) and hence the rank k of the lemon's label amongst the remaining 468 fruits. If she sees less than 2 eaten fruits, she can easily deduce the lemon as well.

Subtask 2

Scoring: If $Y > 9$, score is 0. Otherwise, score is $30 \times \min\left(\frac{30}{X}, 1\right)$.

We are trying to minimise X while eating at most 9 fruits.

Solution 3: $X = 81$**,** $Y = 7$

Constructing b:

Takina sends the labels of the last 9 fruits through the bit string, which requires $9 \times 9 = 81$ bits. The idea is to use the $2^9 = 512$ subsets of these 9 fruits in encoding.

This achieves $X = 81$.

Eating fruits:

If the lemon appears before the last 9 fruits:
There are 491 possible fruits for the lemon. Takina eats a subset of fruits from the last 9 fruits to encode the label of the lemon. There are $2^9 = 512$ possible ways to do this. Now, if the lemon's label is the k-th smallest amongst the first 491 fruits, we eat a subset of fruits in the last 9 so that this subset is the k-th lexicographically smallest subset amongst all 512 subsets.
To distinguish this case from the next, we make sure that Takina always eats at least 2 fruits.
Takina can also eat at most 7 fruits, as $2^9 - \binom{9}{0} - \binom{9}{1} - \binom{9}{8} - \binom{9}{9} = 492 > 491$.

If the lemon is among the last 9 fruits:
Takina eats the fruit immediately after the lemon (or nothing if the lemon is last).

This achieves $Y = 7$.

Recovering lemon:

Chisato uses b to recover the labels of the last 9 fruits. If she sees 2 eaten fruits, she deduces the rank k of the subset of fruits eaten in the last 9 fruits and hence the rank k of the lemon's label amongst the remaining 491 fruits. If she sees less than 2 eaten fruits, she can easily deduce the lemon as well.

Solution 4: $X = 67$**,** $Y = 8$

This is an improved version of Solution 3, which sends over the labels of the last 9 fruits. However, we only made use of the relative rankings of the last 9 fruits. We utilise this fact in Solution 4.

Instead of sending the last 9 labels directly, we construct the *auxiliary tree* of the segment tree over $[1, N]$ for the last 9 labels.

The auxiliary tree is the minimal subset of nodes of the segment tree that contains all pairwise LCA nodes of the 9 chosen leaf nodes. This tree has at most 17 nodes and 16 edges.

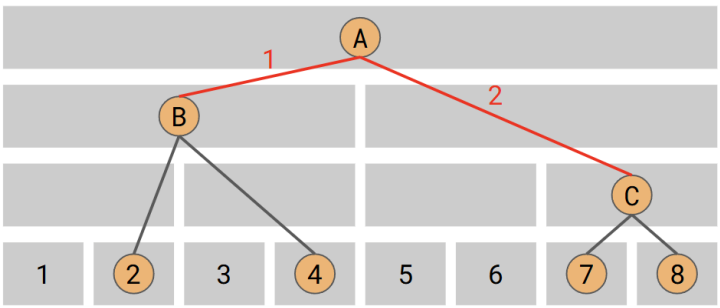


Figure 1: Example auxiliary tree (orange) for indices $2, 4, 7, 8$ on a segment tree over $[1, 8]$.

Takina now wishes to convey this auxiliary tree (along with the height differences of non-leaf edges highlighted in red) over to Chisato. To see why this works, look at **Recovering Lemon** below.

Constructing b:

We encode the auxiliary tree by sending:

- The **structure** of the tree using the up-down sequence of the Euler tour: 32 bits, since the length of an Euler Tour is twice the number of edges.

- The **edge weights** (depth differences) for all edges except those connected to leaves: $8 \times 3 = 24$ bits. ($7 \times 3 + 3$ because there are 7 non-leaf edges and we also need to convey the depth of the highest node)

We would also need to send the **XOR** value of the last 9 labels, using 9 more bits.

This achieves $X = 32 + 24 + 9 = 65$.

Eating fruits:

If the lemon appears before the last 9 fruits:
Same as **Solution 3**, but the rank k of the subset must be exactly equal to the lemon's label. Takina can eat any number of fruits from 0 to 7 because $2^9 - \binom{9}{8} - \binom{9}{9} = 502 > 500$.

If the lemon is among the last 9 fruits:
Takina eats all fruits in the last 9 except the lemon.

This gives $Y = 8$.

Recovering lemon:

If Chisato sees 8 eaten fruits, she uses the XOR value to deduce the lemon's label.

If Chisato sees fewer than 8 eaten fruits, she first recovers the auxiliary tree and the edge weights of non-leaf edges from b.

Then, for each eaten fruit with label F, she can start from the root of the auxiliary tree and trace down which leaf node in the auxiliary tree represents F with the following algorithm.

Algorithm:

1. Suppose Chisato knows that F is in the subtree of node x for some x in the auxiliary tree. (Initially x is the root).
2. Let the left and right children of x in the **original segment tree** be x_{left} and x_{right}. Since Chisato can recover the depth of x in the **original segment tree**, she can deduce if F is under x_{left} or x_{right} in the segment tree.
3. If F is under x_{left}, then F is in the subtree of the left child of x in the **auxiliary tree**.
   If F is under x_{right}, then F is in the subtree of the right child of x in the **auxiliary tree**.

Chisato can repeat this until she deduces which leaf node in the auxiliary tree represents F, hence deducing the rank of F amongst the labels of the last 9 fruits. If she does this for each eaten fruit, she can find the lexicographical ranking k of the subset of fruits eaten amongst all subsets of the last 9 fruits. k will be the label of the lemon.

Solution 5: $X = 49$**,** $Y = 8$

We improve **Solution 4** by taking fewer bits to send the **structure** (refer to Solution 4) of the auxiliary tree.
Imagine that we are building the auxiliary tree, starting with just 9 leaf nodes and working bottom-up. Each step, we merge 2 connected components of leaf nodes by drawing their LCA. Encode this step using the ranking of the rightmost leaf node in the left connected component.

Example (Figure 1 in **Solution 4**):

1. Starting with $[2], [4], [7], [8]$ each as their own components.
2. Add B to the tree by merging $[2], [4]$ to $[2, 4]$. Encode this step as "1". Since 2 is the smallest label.
3. Add C to the tree by merging $[7], [8]$ to $[7, 8]$. Encode this step as "3". Since 7 is the third smallest label.
4. Add A to the tree by merging $[2, 4], [7, 8]$ to $[2, 4, 7, 8]$. Encode this step as "2". Since 4 is the second smallest label.
5. This tree structure is encoded as 132.

Since no step will be encoded into the rank "9", the final encoding will be a permutation of 1 to 8, giving $8! = 40320$ possible tree structures, which is 16 bits.

Constructing b**:**

This achieves $X = 16 + 24 + 9 = 49$.

Eating fruits: Same as Solution 4.

Recovering lemon: Same as Solution 4.

Solution 6: $X = 45$**,** $Y = 8$

Instead of explicitly sending the labels of the last 9 fruits, Takina and Chisato share a precomputed set of *monotone functions* $f : [1, 500] \to [1, 9]$ satisfying $x < y \implies f(x) \leq f(y)$.

When Takina discovers the labels of the last 9 fruits, she finds a function from the shared set that maps each of the 9 labels to a *distinct* output in $\{1, \ldots, 9\}$ (This is so that the function takes each label as inputs and outputs its relative ranking amongst the 9). She sends the index of this function through the bit string.

A monotone function from $[1, 500]$ to $[1, 9]$ corresponds to choosing 8 cutting points from $[1, N-1]$, i.e., splitting $[1, N]$ into 9 segments.

A heuristic to generate a set of functions using dynamic programming and divide-and-conquer is as follows:

$$dp(x, y) = \text{number of functions to map } [1, x] \to [1, y]$$

To calculate $dp(x, y)$, simply let $x/2$ be a cutting point in each function, then add the cases where $x/2$ cannot be a cutting point (i.e. all cutting points are smaller than $x/2$ or all cutting points are larger than $x/2$).

$$dp(x, y) = \sum_{i=1}^{y-1} dp(x/2, i) \cdot dp(x/2, y-i) + 2 \cdot dp(x/2, y).$$

Constructing b:

Computing $dp(500, 9) = 49341224210$, which requires $\lceil \log_2 49341224210 \rceil = 36$ bits.

Similar to **Solution 4**, Takina still needs to send the **XOR** of the last 9 labels.

This achieves $X = 36 + 9 = 45$.

Eating fruits: Same as Solution 4.

Recovering lemon: Same as Solution 4, but for each eaten fruit F, Chisato uses the function identified by the function index to recover the ranking of F amongst the last 9 fruits. The rest is the same.

Solution 7: $X = 44$**,** $Y = 8$

Constructing b:

We improve **Solution 5** by noting that the 8 non-leaf nodes of the auxiliary tree (9 leaves, 17 total nodes) define the auxiliary tree uniquely. These 8 nodes form an unlabelled binary tree. The number of such trees is the 8th Catalan number $C_8 = 1430$. We enumerate all such trees and assign each an index via dynamic programming, requiring $\lceil \log_2 1430 \rceil = 11$ bits.

This achieves $X = 11 + 24 + 9 = 44$.

Eating fruits: Same as Solution 4.

Recovering lemon: Same as Solution 4.

Solution 8: $X = 41$**,** $Y = 8$

Constructing b**:**

This solution presents another way to convey the relative ranking of eaten fruits, amongst the labels of the last 9 fruits. It suffices for Takina to send the *even-ranked* elements among the 9 sorted last labels (the 2nd, 4th, 6th, and 8th smallest labels). There are $\binom{500}{4}$ ways to choose 4 values from $[1, 500]$. Hence these 4 values can be sent using $\lceil \log_2 \binom{500}{4} \rceil = 32$ bits.

This achieves $X = 32 + 9 = 41$.

Eating fruits: Same as Solution 4.

Recovering lemon: Same as Solution 4, but for each eaten fruit F, Chisato uses the 2nd, 4th, 6th and 8th smallest labels amongst the last 9 to recover the ranking of F amongst the last 9 fruits. The rest is the same.

Solution 9: $X = 31$**,** $Y = 8$

Constructing b**:**

We improve on **Solution 7**. Instead of sending the tree structure and edge weights separately, we enumerate *all possible weighted unlabelled binary trees* with 8 nodes and maximum height 8, and assign each a single index, which is doable through dynamic programming. There are 7303164 distinct weighted tree structures, requiring $\lceil \log_2 7303164 \rceil = 23$ bits.

Constant optimisation:
Takina does not need to send over all 9 bits of the **XOR** of the labels of last 9 fruits. Takina just needs to send the last 8 bits. This is because if we define the underlying segment tree in the auxiliary segment tree to split $[1, 500]$ into $[1, 256]$ and $[257, 500]$, then we can determine how many of the 9 labels are ≤ 256 and how many are > 256, allowing us to deduce the first bit of the XOR. We can then reconstruct the whole XOR value.

This achieves $X = 23 + 8 = 31$.

Eating fruits: Same as Solution 7.

Recovering lemon: Same as Solution 7.

Subtask 3

Scoring: Score is $60 \times \min\left(\frac{20}{X+Y}, 1\right)$.

Solution 10: $X = 18$**,** $Y = 42$

Constructing b:

Takina locates the last 43 fruits. She partitions them into two **buckets**: the 21 smallest labels and the 22 largest labels. Call these buckets A and B.

In the bit string, Takina sends:

- The XOR of labels in A, XOR_A
- The XOR of labels in B, XOR_B

This achieves $X = 18$.

Eating fruits:

If the lemon appears before the last 43 fruits:
There are 457 possible fruits for the lemon. Takina encodes the lemon's label using two quantities:

- The rank r_A of the uneaten fruit in bucket A: 21 values (1 to 21).
- The rank r_B of the uneaten fruit in bucket B: 22 values (1 to 22).

There are $21 \times 22 = 462$ possible encodings. Now suppose that the lemon's label is the k-th smallest in the first 457 fruits. Takina will then eat fruits so that the pair (r_A, r_B) is the k-th lexicographically smallest pair amongst all possible pairs.

If the lemon is among the last 43 fruits:
Takina eats all 42 fruits except the lemon.

This achieves $Y = 42$.

Recovering lemon:

Chisato knows the values XOR_A and XOR_B from b.

If Chisato sees 42 fruits eaten, she can first find the XOR of labels of all fruits in the last 43 fruits with XOR_A and XOR_B, then recover the label of the lemon.

If Chisato sees 41 fruits eaten, she can recover the label of the uneaten fruits in buckets A and B using XOR_A and XOR_B, then recover their ranks r_A and r_B. This gives the rank k of the lemon's label amongst the other 457 fruits.

Solution 11: $X = 18$**,** $Y = 30$

Constructing b:

Takina locates the last 31 fruits. She sorts them by label and picks the median label as a **separator**, splitting the remaining 30 into two **buckets**: the 15 smallest labels (bucket A) and the 15 largest labels (bucket B).

In the bit string, Takina sends:

- The label of the median (9 bits).
- The XOR of all 31 labels (9 bits).

This achieves $X = 18$.

Eating fruits:

If the lemon appears before the last 31 fruits:
There are 469 possible fruits for the lemon. Takina encodes the lemon's label using three quantities:

- The number of fruits eaten in bucket A: ranges from 0 to 15, giving 16 values.
- Whether the separator is eaten: 2 values.
- The number of fruits eaten in bucket B: ranges from 0 to 15, giving 16 values.

This conveys $16 \times 2 \times 16 = 512 \geq 500$ possibilities, which suffices. However, Takina must limit herself to eating at most 29 fruits to deconflict with the other case. This still works as there are now $512 - 1 - 3 = 508 > 500$ possibilities. Takina maps each of $[1, 500]$ to a unique possibility (this mapping is shared with Chisato before the game begins). She then looks at the label of the lemon and eats the corresponding number of fruits from each bucket and corresponding decision to eat the separator or not.

If the lemon is among the last 31 fruits:
Takina eats all 30 fruits except the lemon.

This achieves $Y = 30$.

Recovering lemon:

Chisato knows the median label and the XOR of all 31 labels from b. Using the sorted uneaten list, she can determine which of the 31 fruits were eaten and which remain. She counts how many fruits were eaten from bucket A, whether the separator was eaten, and how many were eaten from bucket B. From these three quantities she recovers the encoded value and hence the lemon's label. If 30 fruits were eaten, the lemon is the sole remaining fruit from the last 31.

Solution 12: $X = 27$**,** $Y = 14$

We extend Solution 11 from 2 buckets to 3 buckets with 2 separator elements.

Constructing b**:**

Takina locates the last 15 fruits. She sorts them by label and picks 2 median labels as **separators**, splitting the remaining 13 into three **buckets** of sizes $5, 4, 4$ (buckets A, B, C).

In the bit string, Takina sends the labels of the 2 separators (18 bits) and the XOR of all 15 labels (9 bits).

This achieves $X = 27$.

Eating fruits:

Same idea as Solution 11, but with five quantities: the number of fruits eaten in each of the 3 buckets and whether each of the 2 separators is eaten. This conveys $6 \times 2 \times 5 \times 2 \times 5 = 600 \geq 500$ possibilities, which suffices.

If the lemon is among the last 15 fruits, Takina eats all 14 except the lemon.

This achieves $Y = 14$.

Recovering lemon:

Same idea as Solution 11, but Chisato counts five quantities instead of three.

Solution 13: $X = 15$**,** $Y = 17$

Main idea: Takina wishes to communicate the exact labels of the last 9 fruits to Chisato, by eating fruits instead of through the bit string b.

To do this, suppose the last 9 fruits have labels $x_1, x_2, \ldots, x_9$. Takina wishes to eat the fruits with labels $x_1 - 1, x_2 - 1, \ldots, x_9 - 1$, so that Chisato can deduce the exact labels of the last 9 fruits.

However, this requires 2 conditions to be met:

- **Condition 1:** We cannot have 2 consecutive labels in $x_1, x_2, \ldots, x_9$. (eg. we cannot have both 254 and 255 amongst them)
- **Condition 2:** We cannot have any of $x_1, x_2, \ldots, x_9$ being 1, as otherwise $x_j - 1 = 0$ for some j.

To ensure these conditions are met, Takina and Chisato can share a family of remapping functions $f_1, f_2, \ldots, f_k$. Where for each i, $f_i(1), f_i(2), \ldots, f_i(500)$ is a permutation of 1 to 500. Then, Takina simply needs to convey the index t of a function so that $f_t(x_1), f_t(x_2), \ldots, f_t(x_9)$ satisfy **Conditions 1 and 2**. Now, we want to minimise the size of this function family, k, while guaranteeing a valid choice for t.

Consider a graph with 500 nodes numbered 1 to 500. We can represent each remapping function f_i as a directed path with 499 edges that visits all 500 nodes, where there is a directed edge from $f_i^{-1}(l)$ to $f_i^{-1}(l+1)$ for each $1 \leq l \leq 499$ (where f_i^{-1} is the inverse of f_i).

Now, **Condition 1** translates to "the path represented by f_t cannot contain an edge from x_i to x_j, for all x_i, x_j in $x_1, x_2, \ldots, x_9$". **Condition 2** translates to "the path cannot begin at one of $x_1, x_2, \ldots, x_9$."

We claim that $k = 46$ functions is enough, if we choose them to be 46 edge-disjoint paths, each beginning at a different node. In order to satisfy **Condition 1**, each of the $\binom{9}{2} = 36$ $x_i - x_j$ edges can invalidate at most 1 path, giving us 10 paths left. Since each of these 10 paths start at a different node, one of them is guaranteed to satisfy **Condition 2** as well.

Now, suppose we have a valid function f_t. Let $y_1, y_2, \ldots, y_9$ be the fruits such that $f_t(y_j) = f_t(x_j) - 1$ for each $1 \leq j \leq 9$.

Constructing b**:**

Takina sends over the index of the valid remapping function t, as described above. Since t is at most 46, this takes 6 bits.

She additionally sends the XOR of these 18 values:

- $x_1, x_2, \ldots, x_9$
- $y_1, y_2, \ldots, y_9$

This achieves $X = 6 + 9 = 15$.

Eating fruits:

Takina always eats each of the fruits with labels $y_1, y_2, \ldots, y_9$.

If at any point, she discovers that the lemon is in one of $y_1, y_2, \ldots, y_9, x_1, x_2, \ldots, x_9$, she eats all 17 of the other fruits.

Otherwise, Takina will then only eat fruits amongst the last 9, in a similar fashion as **Solution 4** while ensuring she eats at most 16 fruits.

This achieves $Y = 17$.

Recovering lemon:

Chisato recovers the function f_t by obtaining t from bit string b.

If Chisato sees 17 fruits eaten, she uses the XOR of the 18 values to find the label of the lemon.

If Chisato sees fewer than 17 fruits eaten, she observes which fruits were eaten and, knowing the function f_t, deduces which fruits are in the last 9. She then decodes the subset to recover the lemon's position, similar to **Solution 4**.

Solution 14: $X = 14$**,** $Y = 17$

We improve **Solution 13** by reducing the number of required matchings k from 46 to 30. Use similar logic and definitions as **Solution 13**.

Key observation: Consider 30 remapping functions of the form $f_i(x) = k_i \times x \pmod{500}$ for odd values of k_i coprime to 500. Each such remapping defines a directed path where each edge connects an even node with an odd node.

Hence, the intersection of the clique formed by $x_1, x_2, \ldots, x_9$ with these 30 remapping functions is a bipartite graph, and a bipartite graph on 9 nodes has at most 20 edges (Proof: if the bipartite graph splits the 9 nodes into x nodes and y nodes, then there are at most xy edges, maximised at $4 \times 5 = 20$).

Thus, we have 10 functions left that satisfy **Condition 1**. Using similar logic as **Solution 13**, one of them will also satisfy **Condition 2**.

Constructing b:

Same as Solution 13, but $k = 30$ instead of 46 now.

This achieves $X = 5 + 9 = 14$.

Eating fruits: Same as Solution 13.

Recovering lemon: Same as Solution 13.

Solution 15: $X = 4$**,** $Y = 17$

Main idea: Takina and Chisato will share a set of pairings that split $1, 2, \ldots, 500$ into 250 pairs, then Takina will eat the corresponding fruit in x_i's pair to communicate that x_i is in the last 9 fruits.

However, this requires the following condition to be met:

- **Condition 1:** no pair (p, q) satisfies that both p and q are amongst the last 9 fruits.

In a similar fashion as **Solution 13**, we need k sets of pairings so that at least one of them is valid (i.e. satisfies **Condition 1**).

We claim that $k = 9$ is sufficient. Consider the following 9 pairings (labels are taken mod 500, i.e. 501 is the same as 1, 502 is the same as 2,. . .):

- $(2m, 3m), (4m, 5m), \ldots, (500m, m)$, $m = 1$
- $(m, 2m), (3m, 4m), \ldots, (499m, 500m)$, $m = 1$
- $(m, 2m), (3m, 4m), \ldots, (499m, 500m)$, $m = 3$
- $(m, 2m), (3m, 4m), \ldots, (499m, 500m)$, $m = 7$
- $(m, 2m), (3m, 4m), \ldots, (499m, 500m)$, $m = 13$
- $(m, 2m), (3m, 4m), \ldots, (499m, 500m)$, $m = 27$
- $(m, 2m), (3m, 4m), \ldots, (499m, 500m)$, $m = 53$
- $(m, 2m), (3m, 4m), \ldots, (499m, 500m)$, $m = 107$

- $(m, 2m), (3m, 4m), \ldots, (499m, 500m)$, $m = 213$

Notice that in these 9 sets of pairings, the differences between the two labels in each pair (mod 500) are $m = 1, 1, 3, 7, 13, 27, 53, 107, 213$ respectively. Notice that from the 3rd term onwards, the sequence is superincreasing ($3 > 1 + 1,\ 7 > 3 + 1 + 1,\ 13 > 7 + 3 + 1 + 1, \ldots$).

Lemma 5.2. *For any 9 distinct elements from* $[1, 500]$*, at least one of the 9 families is* valid*: all 9 elements appear in different pairs.*

Proof sketch. Suppose not, let us find a contradiction. Let the 9 elements be $x_1, x_2, \ldots, x_9$. Then each of the 9 set of pairings have 1 pair that includes 2 numbers from $x_1, x_2, \ldots, x_9$. Consider the graph formed by 9 nodes $x_1, x_2, \ldots, x_9$.

For each of the 9 sets of pairings, we pick exactly 1 $x_i - x_j$ edge such that (x_i, x_j) is a pairing in that set, and weight it with the corresponding m value $(1, 1, 3, 7, 13, 27, 53, 107, 213)$. This weight is **equal to the difference between** x_i **and** x_j **(mod** 500**)**.
We hence obtain 9 edges $E_1, E_2, \ldots, E_9$ with weights $w_1, w_2, \ldots, w_9$ that is a permutation of $1, 1, 3, 7, 13, 27, 53, 107, 213$.

Now, since there are 9 edges and 9 nodes, there is a cycle of length at least 3. Let the weights on this cycle be $a_1, a_2, \ldots, a_t$, $t \geq 3$.
If we walk along this cycle, starting from some node until we end up back to that node, then the property that the weight of an edge equals the different of endpoints (mod 500) implies that

$$\pm a_1 \pm a_2 \pm \cdots \pm a_t = 500c$$

for some choice of signs and integer c.

If $c = 0$, then assume for convenience that a_1 is the largest weight amongst them. The equation is impossible as $a_1 > a_2 + a_3 + \cdots + a_t$ by the superincreasing property.

If $c \neq 0$, then the equation is also impossible as $500 > 1+1+3+7+13+27+53+107+213 > a_1 + a_2 + \cdots + a_t$

A contradiction hence arrives. □

By **Lemma 1.2**, these 9 sets of pairings would suffice.

Constructing b**:**

Takina finds a valid set of pairings. She sends index i of the set of pairings in the bit string using $\lceil \log_2 9 \rceil = 4$ bits.

This achieves $X = 4$.

Eating fruits:

Let the last 9 fruits have labels $x_1, x_2, \ldots, x_9$. Let $y_1, y_2, \ldots, y_9$ be the other label in the pair respectively.

Takina always eats each of the fruits with labels $y_1, y_2, \ldots, y_9$.

If at any point, she discovers that the lemon is in one of $y_1, y_2, \ldots, y_9, x_1, x_2, \ldots, x_9$, she eats all 17 of the other fruits.

Otherwise, Takina will then only eat fruits amongst the last 9 as follows:

Collect each pair (x_i, y_i), let p_i and q_i denote the smaller and larger of x_i and y_i respectively.

Order the pairs $(p_1, q_1), (p_2, q_2), \ldots, (p_9, q_9)$ in increasing order of p_i to obtain the new sequence of pairs $P_1, P_2, \ldots, P_9$. Now, each pair P_i has its y value eaten.

Suppose the lemon's label is the k-th smallest amongst the 482 remaining fruits. Choose a subset S of $1, 2, \ldots, 9$ of size at most 7 so that its lexicographical rank amongst all subsets of size at most 7 is k.

For each $i \in S$, eat the x value in the pair P_i.

This achieves $Y = 17$.

Recovering lemon:

Chisato recovers the set of pairings from b.

If she sees 17 eaten fruits, she can deduce all 9 pairs involving $x_1, x_2, \ldots, x_9$ and hence be able to deduce the lemon's index.

If she sees less than 17 eaten fruits, she can deduce all 9 pairs involving $x_1, x_2, \ldots, x_9$, then recover the rank k of the lemon's label amongst the other 482 fruits.